

Introduction	2
Assumptions.....	2
git knowledge.....	2
a useful git alias (put in .gitconfig)	2
git checkout -b	2
git cherry-pick	3
the 'commitish" concept.....	3
tags.....	3
The Patch Stack Model.....	4
Getting started.....	5
Familiarization.....	5
Starting Patch List	6
Git Tree setup.....	6
The Process: repeat until the patch stack is empty	6
git cherry-pick	6
Conflict Resolution	6
Annotate Your Patch Description	6
Fix Compile Failures(if any)	6
Fix Undefined Symbol Issues.....	7
If Things Seem Irretrievably Broken.....	7
Things to Check Before Moving on to the Next Patch	7
Check Your Commit.....	7
Again, Make Sure Your Commit Builds	7
Minimal Sanity Testing.....	8
@@Before Declaring A Backport "Customer Ready"	8
Finis	8
Appendices.....	8
Properly Documenting Your Backported Commit in the Commit Message	8
Upstream Ancestry	8
Deviations from Upstream Ancestor	9
Testing.....	9
Identifying the Patch That Provides a Symbol(or #define)	9
Fix Link Failures Due To Unsatisfied Symbol References Introduced by the Current Patch.....	10

Debugging Suggestions 11

Definitions 11

 Upstream Patches 11

 RC Patches..... 11

 Public Patches 11

Comments/Additions, 12

Hope for a Soon to be Better Future 12

Introduction

This page is a high level discussion of a reasonably repeatable process for backporting. The "reasonably repeatable" wording is because there are decisions the engineer which must make that may be made differently by different people or the same person as they gain experience.

It's a team project. If we notice a theme in comments added in the comments section, we may create a new section

Assumptions

- **Only upstream patches are backported unless you have specific authorization.**
- **Every commit in a series delivered to a customer builds and the resultant kernel boots.**
 - This was true when the patches were upstreamed and we should not break that unless we want to look incompetent in front of the customer.

git knowledge

a useful git alias (put in .gitconfig)

- - - lg = log --pretty=tformat:'%<(10) %h %<(12) %an : %s'
 - example output: aadf62a88131 Stephane Eranian : perf/x86/amd: Make Zen3 branch sampling opt-in
 - short sha id, author, commit subject

git checkout -b

- -

- Know how to use this to save intermediate work states.
 - for example, if you're working with **mybranch.1** and you've just cherry-picked a patch and it builds with no unresolved symbols, save that state before proceeding by:
 - `git checkout -b mybranch.2`
 - It's advisable to note the state you think

git cherry-pick

- -
 - A well known git command that allows you to "copy" a commit via sha-1 id from wherever to your current branch

the 'commitish' concept

- -
 - if <sha> is aadf62a88131
 - `git show aadf62a88131` => shows you the patch
 - `git checkout aadf62a88131` => checks out a **complete tree** as of that commit
 - `git show ca5b7c0d9621:arch/x86/events/amd/brs.c` => show the entire brs.c file as of that commit
 - `git diff <commit-a>.. => show the difference between two commits`

tags

- -
 - linux releases are identified by tags
 - v6.2-RC1: a release candidate
 - a patch in an RC is "public" but it's not "upstream" until it shows up in a "dot release"
 - v6.2: a "dot release"
 - after a release candidate chain is judged good enough
 - `git log`
 - `git worktree`
 - This is potentially useful as it allows you to have a complete tree
 - rebase and autosquash
 - These allow to keep an incremental, traceable history of changes until it's time to release to the customer
 - `git commit --fixup`

- git rebase ... autosquash
- git format-patches
- git send-mail
- add remote
- backing up your work
 - use git add remote to reference a backup repo that you can push branches to
- mbox format
- cscope
 - an ancient tool that allows you to lookup where symbols are defined, used and other things.
- KConfig files
 - patches here will show you new kernel config options you'll need to remain aware of

The Patch Stack Model

- Let's suppose
 - **notation**
 - **<prereq_1_of_patch_1>** is a symbolic representation of "the SHA-1 that the patch with this symbolic name has"
 - you have a patch list to backport BPS = [<patch-1>, <patch-2>, <patch-3>]
 - you cherry pick the first patch to your branch and resolve any conflicts
 - be careful and thoughtful about resolving conflicts this is a **major error opportunity**.
 - increasingly complex resolution requirements are a primary sign that something is going wrong.
 - experience will tell you if it's legitimate complexity or a sign of an error
 - and compile successfully **except function_one()** and **function-two()** are undefined
 - using cscope, or some similar tool, you determine that these functions are in commits **<prereq_1_of_patch_1>** and **<prereq_2_of_patch_1>**
 - before we know what order the patches have relative to each other(**error opportunity**), we need consider
 - what release contains each patch(**error opportunity**)
 - see git tag --contains and consider the earliest dot release containing the patch
 - it's a **concern** if the patch is **not** in in the latest kernel release
 - git investigation to see if it was reverted or dropped is required before proceeding
 - **latest kernel ordering**
 - if both patches are present in the latest kernel, then they should be applied in that order

- if #1 is on line N of the git lg <release-tag> output and #2 is on line M and M > N then apply in the order M, N
 - recall that in the git log output the top commit is applied last and the bottom first
- **inter-release ordering**: if the patches are in different releases, push them on the stack so the patch for the one to be applied next is on top
 - if this conflicts with **latest kernel ordering**, there is an issue that must be understood
 - it is not sufficient to just assume **latest kernel ordering** is the ultimate truth .
 - look at the git lg -1 output for the first release that contains both patches. That may (**error opportunity**) be the proper ordering between them.
 - Refactoring may have occurred farther down the road which results in the relative ordering being different in the latest kernel.
 - What's optimal depends on the ordering of
 - your target release
 - the source release of the patches
 - any refactoring that may have changed the order
 - **intra-release ordering**: if both patches are in the same release, then that is **probably** the right order
 - Since both patches are needed the cost of being wrong is just more error prone conflict resolution than would otherwise be needed.
 - so in our imaginary example, the patch stack now looks like BPS = [<prereq_1_of_patch_1>, <prereq_2_of_patch_1>, <patch-1>, <patch-2>, <patch-3>]
 - this means that we need to use **git reset – hard HEAD~<some-number>** to remove anything we applied to do with patch 1 since the conflict resolution may differ depending on the contents of the prerequisites.

Getting started

Familiarization

Things may help include

- Reviewing your patch series to get an idea of the scope of files it touches
- Look in the kernel's Documentation/ directory for documentation touching on things related to your patch series.

Starting Patch List

- you'll start with some sort of patch list (ex: Genoa Essential Patches)
- your initial patch stack is <first patch> ... <last patch>
- verify that all patches are upstreamed(already done for for you on the Genoa Patch List but beware of links from that page to other patch lists(ex: BRS)

Git Tree setup

- Clone a git tree that has most current and past releases for analyzing patch history.
- Use git add remote to connect your "backup repo" to the repo you just cloned.
 - I'll use the alias "backup" here for the remote

The Process: repeat until the patch stack is empty

git cherry-pick

- first do **git status** and make sure
 - no cherry-pick is in progress
 - you see "nothing to commit, working tree clean"
- **git cherry-pick** <prereq_1_of_patch_1>

Conflict Resolution

- At this point there are several possibilities:
 - There were **no conflicts** and the patch is no applied
 - git status => see "nothing to commit, working tree clean"
 - git lg -1 => subject line of the commit you just cherry-picked
 - **Changes to be committed**
 - If there was any conflict, it was resolved automatically
 - Both modified
 - look for the "<<<<<<<<<" and ">>>>>>>>" and resolve the conflict(**error opportunity**).
- when all conflicts are resolved(and this is confirmed by git status)
 - **git cherry-pick --continue**

Annotate Your Patch Description to Refer to Upstream Ancestor

- Use **git status** to **make sure** all conflicts are resolved.
- then use git commit --amend to modify the commit message to describe upstream ancestry and deviations from the upstreami patch per [Properly Documenting Your Backported Commit in the Commit Message](#) .
 - ex: **commit** <sha1> **upstream**

Fix Compile Failures(if any)

Common examples of things that can get missed after you've been resolving conflicts for hours include

- mismatch of #if/#else/#endif

- dropped braces or semicolons
- **make** -i \$MANY_CORES
- see the error messages and, for now ignore undefined symbols and focus on other issues
- resolve the issues
- **git status** => verify all clear
- **git cherry-pick --continue**
- **git lg -1**
- assume you see: aadf62a88131 first_name last_name : prereq_1_of_patch_1
 - **aadf62a88131** is the sha-1 of the deconflicted, cherry-picked commit
"prereq_1_of_patch_1"

Fix Undefined Symbol Issues

- see appendix

If Things Seem Irretrievably Broken

- Consult your notes to find the last patch where you had no underfined symbols(or use git bisect to figure that out with brute force and time)
- git checkout -b your_branch.N+1 // increment your branch number
- git reset --hard <sha of the patch after the last one that had no undefined symbols>
 - **be very careful with this^^^^^^^^^^**.
- re push all the the sha ids of the patches whose backported versions you deleted in the previous

Things to Check Before Moving on to the Next Patch

- **caution: incomplete list** - depending on your context you may need to do more or different things

Check Your Commit

- Look at your commit
 - git lg -1
 - see the expected subject line
 - do **git show <sha1>** and ensure nothing in there surprises you
 - diff your commit.vs. the upstream source commit, the contents should not be a surprise
 - **git diff <sha1-that-you-just-commited> >diff.mine**
 - **git show <sha-of-upstream-commit> >diff.upstream**

Again, Make Sure Your Commit Builds

The term "builds" means more than just **make** in the kernel tree works without error. For example, you have a set of commits that touches the kernel and the **perf** util, you must make sure:

- the kernel builds
- the perf utility builds

Minimal Sanity Testing

The goal here is to be quick and catch egregious problems. Regression testing is not usually appropriate at this stage. Minimal sanity testing, depending on what you're doing can include

- boot the kernel
 - see something that indicates the feature you're working on is operating correctly.
 - ex: the perf counter your patches are adding is present and reported correctly.
- boot a KVM guest
 - See that the feature works properly in the guest(may be different than host operation)
- Run tests described in the upstream commit message.
- Run "well known" tests like **perf test**.
- Run any new tests you've written (ex: kvm unit tests)

@@Before Declaring A Backport "Customer Ready"

- Recheck that all your individual patches conform to what's specified in [Properly Documenting Your Backported Commit in the Commit Message](#).
- Write up a one pager to be put on the [AMD Backporting Page](#) .
 - There must be a section in the one pager documenting the testing you've done so that a review/customer
- Get approval from at least one review designated by a responsible AMD manager.
- Get approval from a responsible AMD manager.
- More ontent coming soon as suggested.

Finis

- delete "most" of the intermediate branches after acceptance testing is completed.

Appendices

Properly Documenting Your Backported Commit in the Commit Message

Upstream Ancestry

note: in this appedix, this highlight is used to indicate things that should be done literally so that scripts can parse easily.

Annotate all patch commit messages per:

<https://www.kernel.org/doc/html/latest/process/stable-kernel-rules.html#option-3>

Note the upstream commit ID in the changelog of your submission with a separate line above the commit text, like this:

commit <sha1> **upstream**

This is essential for internal AMD patch review and customer review via automated tools.

Deviations from Upstream Ancestor

If the submitted patch deviates from the original upstream patch (for example because it had to be adjusted for the older API), this must be **very clearly** documented and justified in the patch description. Some general guidelines:

"very clearly" means no internal or customer review of your patches should look at a diff hunk after reading your explanation and wonder "what does this do?"

Use a format like:

[Upstream Deviations]

+ deviation 1: very short description. (like a patch subject)

+ deviation 2: very short description

+ ...

+ deviation N: very short description

[deviation 2]

extended multiple paragraph description if needed

Testing

- Sometimes, when you're backporting a series of patches, there's not a lot new to test until you get the last one done. In that case the testing section should look like:

[Testing: kernel built, booted and verified build date via uname -a]

- General guidelines are:
 - Mention how all build products affected by the patch were verified. (ex: kernel and util/perf: built, booted, and verified running latest binary.
 - This section must be sufficiently detailed for
 - a reviewer or customer to **duplicate your testing** if there is a problem and the patch series needs to be bisected.
 - a reviewer or customer to say "that's not adequate testing, do this too"

Identifying the Patch That Provides a Symbol(or #define)

- Undefined symbols mean that the files touched by the patch that you just cherry-picked reference a function that is (yet) present .
- A short outline of how to resolve these, which may need to be modified in specific scenarios(**error opportunity**).

- Use the worktree you created to browse the latest kernel tree(ex: v6.7)
- **tactic 1**
 - Use cscope or some tool like it to find the file defining the symbol
 - **git blame** path-to-file
 - find the symbol you're concerned with
 - the left column of the git blame output is the short sha-1 is the last commit to modify this declaration]
- **tactic 2**
 - **git lg -S** the-symbol-you-want v6.6
 - v6.6 is just a stand in for "latest kernel you have"
 - you will get a list in order from most recent to least recent of commits that added or removed references to the symbol
 - there are many options to git log that you can use for various effects but **-G** is important because **-S** will only find commits where the number of occurrences changed and miss commits that modified the use of the symbol(ex: changed function arguments)
 - common one is to pipe the output of your git log command to grep to, for example, select lines beginning with '+' to find commits that added the symbol
 - you should probably do both tactics and, in the relatively unlikely event they don't give the same answer, understand why before proceeding.

Fix Link Failures Due To Unsatisfied Symbol References Introduced by the Current Patch

There is a lot to potentially say here but, the short story is:

- There is a fundamental choice to be made in approach. Generally, it makes sense to try depth first first. In the case of dept first you need to keep track of a stack of prerequisite patches. In the case of breadth first it's a tree. In either case take careful notes so you always know "I'm cherry-picking path P_N because of symbol s defined in P_{N-1} ."
 - **breadth first:** cherry pick all required symbols that were undefined in the link
 - this can get very complex very fast.
 - you wind up with a tree $O(\text{undefineds_per_level}^{\text{tree_depth}})$ nodes
 - **depth first:** cherry pick only one patch providing one of the undefined symbols
- Repeat until it links with no undefined symbols /* one patch at a time for depth first */
 - Identify patches providing the undefined symbol(**s**) from linker output in your build phase. (see the separate section on that topic)
 - Cherry pick all of those patches
 - build and link
 - now you have a list of patches
 - applied = [...,
<last patch that compiled and linked>,
<P1: patch that referenced undefined symbol **u1**>,

<P₂: provides symbol **u₁**, but references undefined symbol **u₂**>

...

<P_{N-1}: provides **s_{N-1}** references undefined symbol **u_N**>

<P_N: provides **u_N** and does not reference anything undefined>

- now look backward into your notes and start working on next symbol that's undefined

Debugging Suggestions

- Use the upstream version ported from to debug, check if behavior is as expected, that is checkout a commit you applied or cherry picked, build install it see if its behaving as expected i.e. is it still throwing kernel faults or its already working but the port is not?

Definitions

Upstream Patches

- **Upstream patches are in a "dot release".**
- Anything less is less good and you should have approval and make explicit release notes of what you've done and why.

RC Patches

- The concern is if a patch was in X.Y-RC<N> and it was gone by N + k, was it removed because it caused a problem?
 - You need to have a clear understanding of why it went away and these stakeholders are going to be interested
 - Internal reviewers
 - Customer reviewers
 - The understanding must be substantiated
 - by citing lre.kernel.org traffic
 - or maybe a clear statement from the developer
- RC patches won't pass internal review without the above and a customer may not be willing to accept them at all
 - Therefore project management should seek pre-approval before wasting potentially huge amounts of development time backporting things that depend on an RC patch

Public Patches

- Dot release patches and RC patches are public
- Anything you can find on lore.kernel.org or similar lists is public
- Patches from internal AMD trees or on internal AMD mailing lists **ARE NOT PUBLIC.**
 - **Don't use them without approval from an AMD manager.**
 - Do
 - get advice and permission from the Server Operating System Team(Jon Grimm)
 - think about Palmida scan
 - think about OSRB submission
- **Don't, don't, don't share them with a customer** without direction from a Director or VP.

Comments/Additions, ...

-

Hope for a Soon to be Better Future

- A tool to automate a lot if the drudgery is in development